

AI Supo 2 — Topics covered

Q1, Q2 (Sec 6) and Q1 (Sec 7)

★ = 80/20 essentials, learn cold.

Section 6 — Situation Calculus

1.1 The language

Situation calculus encodes a changing world inside first-order logic.

Core ontology

Situation s	A snapshot of the world. s_0 is the initial situation. Situations are objects in the logic, so you can quantify over them.
Fluent	A predicate whose truth value depends on a situation. Last argument is always s . E.g. $AT(Robot, kitchen, s)$, $HAVEARROW(s)$.
Action	A first-class object (<i>not</i> a situation). E.g. $GO(kitchen, hall)$, $SHOOT$.
Result (a, s)	A function returning the situation that results from doing action a in situation s .
Goal (s)	A fluent meaning “the goal is satisfied in s ”.

Static facts about the world (e.g. $ADJACENT(p_1, p_2)$, $WALL(p)$) are *not* fluents — they don’t depend on s .

1.2 Composing actions: Sequence/Do

$SEQUENCE(a, s, s')$ means “executing the action list a from s produces s' ”. Defined recursively:

$$SEQUENCE([], s, s), \quad SEQUENCE([a \mid rest], s, s') \iff SEQUENCE(rest, RESULT(a, s), s').$$

1.3 The frame problem

When action a happens, almost *everything* stays the same — only a tiny fraction changes. Stating “everything else is unchanged” naively requires $O(\text{actions} \times \text{fluents})$ frame axioms. That’s the **frame problem**.

Successor-state axioms (the fix)

For *each* fluent F , write *one* axiom of the form

$$F(\vec{x}, RESULT(a, s)) \iff \underbrace{\Phi^+(\vec{x}, a, s)}_{a \text{ makes } F \text{ true}} \vee \left(F(\vec{x}, s) \wedge \neg \underbrace{\Phi^-(\vec{x}, a, s)}_{a \text{ makes } F \text{ false}} \right).$$

This combines effect axioms *and* frame axioms in a single sentence per fluent. Reasoning becomes tractable.

Common student error: writing one axiom *per action* rather than per fluent. Successor-state axioms close over *all* actions a at once.

1.4 Possibility axioms

For each action A , give one axiom $\text{POSS}(A, s) \iff \dots$ saying when A is applicable in s . This is the precondition.

Recipe for every action

1. One **possibility** axiom: $\text{POSS}(A, s) \iff \dots$
2. One **successor-state** axiom *per fluent* that any action (including A) can affect.

1.5 Planning queries (Q1)

There are *two* ways to phrase “find me a plan” as a logical query.

Form A — explicit situation

$$\exists a \exists s. \text{SEQUENCE}(a, s_0, s) \wedge \text{GOAL}(s)$$

Read: “find an action list a and a situation s such that running a from s_0 produces s , and the goal holds in s .” A constructive proof binds a to a plan.

Form B — situation hidden inside the action list

$$\exists \text{actionList}. \text{GOAL}(\dots \text{actionList} \dots)$$

No explicit s . The goal predicate must now take an action list as argument.

How to make Form B work (the Q1 answer in three moves):

1. Define an auxiliary DO that folds the action list through RESULT starting from s_0 :

$$\text{DO}([], s) = s, \quad \text{DO}([a \mid \text{rest}], s) = \text{DO}(\text{rest}, \text{RESULT}(a, s)).$$

2. Redefine the goal predicate so the situation is *internal* to it:

$$\text{GOAL}(\text{actionList}) \iff \text{GOALHOLDS}(\text{DO}(\text{actionList}, s_0)).$$

Here $\text{GOALHOLDS}(s)$ is the original situation-based goal fluent.

3. Equivalence with Form A: the existentially quantified s is exactly $\text{DO}(\text{actionList}, s_0)$. So Form B holds iff Form A would have with s instantiated to that value. The same successor-state axioms make either form evaluable.

Why bother? The witness from a constructive proof of Form B is *just* the action list — which is what the planner actually wants. Situations are a derived intermediate quantity that Form A forces you to quantify over.

1.6 Wumpus World: Shoot, Move, Orient (Q2)

Constants & helpers (assumed non-fluent unless marked):

- Locations $[x, y]$; orientations $\in \{0, 90, 180, 270\}$; AGENT, WUMPUS constants.
- $\text{LOCATIONTOWARD}([x, y], d)$: the adjacent cell in direction d .
- $\text{LOCATIONAHEAD}(s) = \text{LOCATIONTOWARD}(\text{LOC}(s), \text{ORI}(s))$.
- $\text{WALL}(p)$, $\text{ADJACENT}(p_1, p_2)$: static map predicates.

Fluents: $\text{AT}(\text{AGENT}, [x, y], s)$, $\text{ORIENTATION}(\text{AGENT}, d, s)$, $\text{HAVEARROW}(s)$, $\text{WUMPUSALIVE}(s)$.

Helper fluent (line-of-fire):

$$\text{FACINGWUMPUS}(s) \iff \exists [x, y]. \text{AT}(\text{WUMPUS}, [x, y], s) \wedge \text{INLINEOFFIRE}(\text{AGENT}, [x, y], s).$$

Shoot

Possibility: $\text{POSS}(\text{SHOOT}, s) \iff \text{HAVEARROW}(s)$.

Successor-state for HaveArrow: $\text{HAVEARROW}(\text{RESULT}(a, s)) \iff \text{HAVEARROW}(s) \wedge a \neq \text{SHOOT}$.

Successor-state for WumpusAlive:

$$\text{WUMPUSALIVE}(\text{RESULT}(a, s)) \iff \text{WUMPUSALIVE}(s) \wedge \neg(a = \text{SHOOT} \wedge \text{FACINGWUMPUS}(s)).$$

Move (Forward)

Possibility: $\text{POSS}(\text{FORWARD}, s) \iff \neg \text{WALL}(\text{LOCATIONAHEAD}(s))$.

Successor-state for At:

$$\begin{aligned} \text{AT}(\text{AGENT}, p, \text{RESULT}(a, s)) \iff & (a = \text{FORWARD} \wedge p = \text{LOCATIONAHEAD}(s)) \\ & \vee (a \neq \text{FORWARD} \wedge \text{AT}(\text{AGENT}, p, s)). \end{aligned}$$

Orient (TurnLeft, TurnRight)

$$\text{LEFTTURN}(d) = (d + 90) \bmod 360, \quad \text{RIGHTTURN}(d) = (d - 90) \bmod 360.$$

Possibility: $\text{POSS}(\text{TURNLEFT}, s) \iff \top$, $\text{POSS}(\text{TURNRIGHT}, s) \iff \top$.

Successor-state for Orientation:

$$\begin{aligned} \text{ORIENTATION}(\text{AGENT}, d, \text{RESULT}(a, s)) \iff & (a = \text{TURNLEFT} \wedge d = \text{LEFTTURN}(\text{ORI}(s))) \\ & \vee (a = \text{TURNRIGHT} \wedge d = \text{RIGHTTURN}(\text{ORI}(s))) \\ & \vee (a \neq \text{TURNLEFT} \wedge a \neq \text{TURNRIGHT} \\ & \wedge \text{ORIENTATION}(\text{AGENT}, d, s)). \end{aligned}$$

1.7 Common pitfalls (Sec 6)

- Writing effect axioms only — forgetting the frame part. Use successor-state form so frames are automatic.
- Writing one successor-state axiom per action instead of per fluent.
- Treating WALL, ADJACENT as fluents and writing pointless successor-state axioms for them.
- Forgetting the outer universal $\forall a, s$ when writing the axioms formally.

Section 7 — Heuristics for Planning

2.1 Planning vs search

The four differences (worth knowing cold)

	Search	Planning
State representation	Opaque (black box)	Structured (set of fluents)
Actions	Arbitrary transitions	Precondition + add-list + delete-list
Goal test	Single function	Logical condition over fluents
Heuristic source	Domain-specific, supplied by human	Can be derived from the structure

2.2 Why planning gives heuristics “for free”

In ordinary search the human writes the heuristic. In planning, each operator’s preconditions and add/delete lists are visible to the planner — so the planner can *automatically* construct a relaxed problem and use its optimal cost as h .

The recipe

Drop part of the problem to get an easier (“relaxed”) problem whose optimal cost is a lower bound on the original. Solving the relaxation must be substantially cheaper (preferably polynomial) and the bound must be admissible.

2.3 Relaxation heuristics

(a) Ignore delete lists (delete-relaxation). Drop every action’s delete-list. Nothing ever becomes false; literals only accumulate. The relaxed problem is monotone and solvable in polynomial time. Its length is h^+ , the canonical admissible heuristic. **Used by FastForward (FF) — one of the most successful planners.**

(b) Ignore preconditions. Drop all preconditions. Every action always applies. Optimal relaxed-plan length = minimum number of actions whose add-lists cover the goal (a set-cover problem). Looser than h^+ but easy to compute.

(c) Subgoal independence. Plan for each goal conjunct in isolation, then combine:

- $h_{\max}(s) = \max_g \text{cost}(g)$ — admissible (under-counts).
- $h_{\text{add}}(s) = \sum_g \text{cost}(g)$ — not admissible (double-counts) but often more informative.

2.4 Other useful heuristics

Number of unsatisfied goal conditions remaining. Quick and dirty; not admissible in general but cheap and surprisingly effective on simple problems.

Planning-graph level. The depth at which a goal first becomes reachable in a planning graph is an admissible heuristic (deeper graph = harder problem). Used by Graphplan.

Helpful actions. Restrict the branching factor at each search node to only those actions that appear in the relaxed plan from that node. Trades completeness for speed.

2.5 How to structure the Q1 (Sec 7) answer

A strong answer names at least **three heuristics from different families** and links them back to the question. A good template:

1. State the key idea: planning's structured representation lets heuristics be derived *automatically* from the problem, unlike generic search.
 2. Give the relaxation recipe (drop part of the problem; lower bound on cost).
 3. Name two or three concrete heuristics: at least one relaxation-based (ignore deletes / ignore preconditions / subgoal independence), and at least one structural (planning-graph level / unsatisfied-goals count / helpful actions).
 4. Close by linking back: planning's edge over generic search is precisely this ability to compute its own heuristic.
-

The 80/20 cheat-sheet (memorise this page)

Sec 6 — non-negotiable

- Situations, fluents, actions, $\text{RESULT}(a, s)$ — definitions on the tip of your tongue.
- For every action: **one possibility axiom, one successor-state axiom per affected fluent.**
- Successor-state schema: $F(\vec{x}, \text{RESULT}(a, s)) \iff \Phi^+ \vee (F(\vec{x}, s) \wedge \neg\Phi^-)$.
- Two query forms: $\exists a \exists s. \text{SEQUENCE}(a, s_0, s) \wedge \text{GOAL}(s)$ vs $\exists \text{actionList}. \text{GOAL}(\text{actionList})$.
- Form B works by defining $\text{GOAL}(\text{actionList}) \iff \text{GOALHOLDS}(\text{DO}(\text{actionList}, s_0))$.

Sec 7 — non-negotiable

- Planning vs search: structured state, structured actions, derivable heuristics.
- The relaxation recipe: drop part of the problem; cost of relaxed solution = lower bound on real cost.
- Two relaxations you must know: **ignore deletes** (h^+ , FastForward), **ignore preconditions** (set cover).
- Subgoal-independence: $h_{\max}$ admissible, h_{add} informative.